Deterministic Semantic Token Compaction, Attention-Sink Structural Anchoring, and Hardware-Aware Pareto Economic Routing for Large Language Model Serving

Aniket Agrawal
Google Cloud AI
Google Cloud India Private Limited
Bangalore, Karnataka, India
aniketagrawal@google.com

***Abstract*—The rapid proliferation of enterprise large language model (LLM) serving and autonomous multi-agent swarms has collided with an operational ceiling: the *LLM Token Economics and Attention Wall*. In production deployments across cloud accelerators, up to 35% of ingested tokens represent synthetic subword tokenizer fragmentation artifacts and low-information conversational boilerplate. This token bloat incurs high financial inference costs, induces copy-on-write page duplication on unaligned trailing Key-Value (KV) cache blocks, and causes attention dispersion across Transformer self-attention heads, precipitating silent hallucinations. In this paper, we present the *Tokenomics Compiler* (`PromptVM`), a deterministic, hardware-grounded compilation framework and discrete-choice economic router for LLM serving. Our system: (1) parses structured schemas into Abstract Syntax Trees (ASTs) and transpiles them into subword-minimal serializations over a bounded candidate space ($|\mathcal{S}_{\text{dense}}| \leq 8$) using localized delta-tokenization while preserving both formal JSON Schema validation contracts and task-semantic parameter role disambiguation; (2) performs subclause pleasantry and duplicate-signature span stripping alongside inverse-length-scaled empirical invariant density scoring to prune diffuse conversational verbiage while unconditionally retaining modal and relational invariants; (3) injects uppercase structural landmark anchors with empirical attention affinity boost ($\gamma > 1$), mathematically proving that landmark anchors bound self-attention entropy on dispersive heads ($\mathcal{H}(\mathbf{a}_i) \leq \mathcal{H}_{\max}(i, K, \gamma) < \ln i$ with $\frac{\partial \mathcal{H}_{\max}}{\partial \gamma} < 0$ for all $\gamma > 1$) while co-locating retained constraint predicates to amplify retrieval-head attention by $2.8\times$; (4) enforces physical KV-cache boundary-page sealing ($|\mathcal{P}_{\text{prefix}}| \equiv 0 \pmod{B}$, $B \in \{16, 32\}$) via server-side JAX/Pallas VMEM causal bitmasking and decoupled RoPE logical positions to eliminate multi-branch copy-on-write page duplication on self-hosted Google Cloud TPU v6e Trillium accelerators; and (5) solves a constrained discrete-choice Pareto optimization across heterogeneous model serving tiers (evaluated across internal harness routing tiers Gemini 3.8 Flash, Gemini 3.8 Flash with dynamic test-time reasoning tokens, and Gemini 4.0 Argon, mapped to production Vertex AI Flash and Pro endpoints). Across $N = 1{,}000$ enterprise traces evaluated across 5 distinct operational cohorts (and validated on public BIRD-SQL and BFCL benchmarks), our compiler achieves a 1.11% to 53.47% per-cohort token reduction (17.52% pooled global reduction, saving 81,800 tokens per 1,000 requests; 39.98% on structured prompt cohorts), slashes blended inference invoice costs by 39.44% (up to 88.79% in database queries), and reduces P95 latency by 38.85% against baseline (27.98% against naive regex trimming) while achieving a 99.70% strict single-attempt pre-escalation Pass@1 and 100.00% post-verification workflow completion rate.**

***Index Terms*—Token Economics, Large Language Models, Transformer Attention Sinks, Subword Compaction, PagedAttention, KV-Cache Alignment, TPU v6e Trillium, Pareto Optimization, Vertex AI, Model Routing.**

I. Introduction

The industrial transition from basic conversational chat interfaces to high-throughput autonomous multi-agent swarms has fundamentally transformed cloud inference demand. Enterprise platforms such as Google Cloud Vertex AI, the Google Agent Development Kit (ADK; https://cloud.google.com/vertex-ai/docs/agent-builder), and the Model Context Protocol (MCP; https://modelcontextprotocol.io) orchestrate millions of daily reasoning-and-acting invocations [1], [2] wherein software agents query distributed databases, validate OpenAPI endpoints, inspect source repositories, and reconcile financial ledgers.

Despite rapid advancements in accelerator throughput and distributed inference scaling [3], [4] (spanning Google Cloud TPU v4 supercomputing pods [3] and Cloud TPU v6e Trillium accelerators; https://cloud.google.com/tpu/docs/v6e-overview), enterprise deployments face an acute economic and physical bottleneck: the *LLM Token Economics and Attention Wall*. Unmanaged prompt construction introduces severe systemic inefficiencies across three distinct operational layers:

1) **The Subword Fragmentation Tax:** Standard subword tokenizers—operating either via SentencePiece Unigram subword segmentation in Gemini and Gemma [5]–[7] or iterative Byte-Pair Encoding (BPE) pair merges—decompose unaligned whitespace, camelCase identifiers, and pretty-printed JSON schemas [8] into multiple fragmented subword tokens. In enterprise schemas, up to

Assignee: Google LLC. Target System: Google Cloud Vertex AI, Google Agent Development Kit (ADK), and Cloud TPU v6e Trillium Infrastructure.

35% of ingested tokens represent structural formatting overhead rather than semantic domain entropy.

2) **Attention Dispersion and Hallucination:** Transformer self-attention layers compute softmax-normalized dot products over query-key pairs [9]. When prompts are laden with verbose conversational filler, the attention probability mass disperses broadly across keys, driving the Shannon attention entropy toward its theoretical maximum $\ln i$. This dispersion dilutes attention weights on negative constraints and schema contracts, causing critical parameters to be hallucinated or omitted.
3) **Hardware KV-Cache Boundary-Page Copy-on-Write Duplication:** Production serving runtimes (e.g., vLLM [10], SGLang [11], and JAX/Pallas Block-Attention kernels [12]) manage persistent Key-Value (KV) cache memory in discrete virtual memory pages of block size $B = 16$ or 32 tokens in High Bandwidth Memory (HBM2e). When shared static prompt prefixes terminate at non-page boundaries ($|\mathcal{P}_{\text{prefix}}| \not\equiv 0 \pmod B$), appending branch-specific dynamic suffix tokens into the unsealed trailing prefix block forces copy-on-write (CoW) page duplication and K/V recomputation across concurrent subagent execution branches.

To overcome these deficiencies, we introduce the *Tokenomics Compiler* (`PromptVM`), a deterministic hardware-aware systems framework that bridges the semantic instruction layer with physical accelerator memory topologies. Rather than relying on fragile, heuristic prompt engineering, our framework formalizes a five-stage compilation and routing pipeline: (1) Tokenizer-Aware AST Transpilation, (2) Information-Theoretic Entropy Compaction, (3) Attention-Sink Structural Landmark Anchoring, (4) Hardware Paged-Cache Block Boundary Alignment, and (5) Discrete-Choice Pareto Economic Routing.

The primary technical contributions of this work are as follows:

- We formalize **AST Subword Merge Optimization** over OpenAPI and JSON schemas [8] across a bounded $2^3 = 8$-candidate AST space using localized delta-tokenization, eliminating synthetic tokenization artifacts under formal validation-contract equivalence while preserving task-semantic parameter role disambiguation.
- We define **Sub-Clause Span Stripping and Invariant Density Scoring** ($\mathcal{D}_{\text{inv}}(\tilde{c}_k)$) with inverse-length threshold scaling and polarity-aware relational predicate signature extraction ($\text{Sig}(c_k)$) to prune conversational noise while unconditionally retaining novel modal and relational constraints regardless of clause length.
- We prove **Theorem 1**, demonstrating that uppercase structural landmark anchors acting as artificial attention sinks with affinity boost $\gamma > 1$ strictly bound query self-attention entropy on dispersive heads ($\mathcal{H}(\mathbf{a}_i) \le \mathcal{H}_{\max}(i, K, \gamma) < \ln i$) with $\frac{\partial \mathcal{H}_{\max}}{\partial \gamma} = -\frac{K(i-K)\ln\gamma}{Z_i^2} < 0$, where $Z_i = K\gamma + (i - K)$, while physical co-location boosts retrieval-head constraint attention by $2.8\times$.
- We co-design a **Client-Server Paged-Cache Topologist** that aligns static prefixes to physical page boundaries ($|\mathcal{P}_{\text{prefix}}| \equiv 0 \pmod B$) with server-side Pallas VMEM causal bitmasking and logical RoPE/window decoupling to seal boundary pages for zero-copy multi-agent sharing on self-hosted Cloud TPU v6e clusters.
- We construct a **Discrete-Choice Pareto Economic Router** that jointly optimizes model tier selection across heterogeneous serving tiers (Gemini 3.8 Flash, Gemini 3.8 Flash-Thinking, and Gemini 4.0 Argon mapped to Vertex AI Flash/Pro endpoints) and dynamic test-time reasoning tokens via an exact closed form.
- We evaluate our system across 5 seeds on $N = 1{,}000$ enterprise traces spanning 5 cohorts and 400 public benchmark tasks (BIRD-SQL [13] and BFCL [2]), demonstrating a 1.11% to 53.47% enterprise token reduction (17.52% pooled mean, 39.98% on structured cohorts), 39.44% net cost savings (up to 88.79% in SQL), 99.70% strict single-attempt Pass@1, and 100.00% post-verification workflow completion.

II. Background and Systems Architecture

A. Transformer Self-Attention Physics and Dispersion

In standard multi-head self-attention, during the prompt prefill phase over a sequence of N tokens, the attention weight matrix at layer l and head h is:

$$\mathbf{A}^{(l,h)} = \text{softmax}\left(\frac{\mathbf{Q}^{(l,h)}(\mathbf{K}^{(l,h)})^T}{\sqrt{d_{\text{head}}}} + \mathbf{M}\right) \in \mathbb{R}^{N\times N} \quad (1)$$

(omitting layer and head superscripts (l, h) below for brevity), where d_{head} denotes the key projection dimension, and $\mathbf{M} \in \{0, -\infty\}^{N\times N}$ is the causal attention mask enforcing $M_{ij} = 0$ for $j \le i$ and $M_{ij} = -\infty$ for $j > i$ in decoder-only autoregressive architectures (including Gemini and Gemma). Consequently, for any prefill query position $i \le N$ (or subsequent autoregressive decode step $i = N + t$ attending over the i accumulated context keys), query token i can attend only to antecedent positions $1 \le j \le i$, and its attention vector $\mathbf{a}_i = (A_{i1}, \ldots, A_{ii})$ lies on the restricted simplex Δ^{i-1} where $\sum_{j=1}^{i} A_{ij} = 1$. The Shannon entropy of this distribution is:

$$\mathcal{H}(\mathbf{a}_i) = -\sum_{j=1}^{i} A_{ij} \ln A_{ij} \quad (2)$$

When the input context consists of diffuse natural language, query-key affinities exhibit low variance, causing the softmax distribution to approach the uniform distribution with maximal theoretical entropy $\mathcal{H}_{\text{diffuse}} \le \ln i$. While initial sequence tokens (such as `<bos>` at $j = 1$) act as natural emergent sinks [14], unstructured text dilutes the attention weights allocated to sparse critical constraints (e.g., `status = 'ACTIVE'`), decaying as $\mathcal{O}(1/i)$ and precipitating retrieval failure and hallucinations. In PromptVM's compiled token layout, the shared static prefix $\mathcal{P}_{\text{prefix}}$ (containing the static system preamble and Stage 1 transpiled schema S^* at indices $\{1, \ldots, |\mathcal{P}_{\text{prefix}}|\}$) and optional Stage 4 boundary padding $\mathcal{I}_{\text{pad}}$ (of size Δ_{frag} on TPU

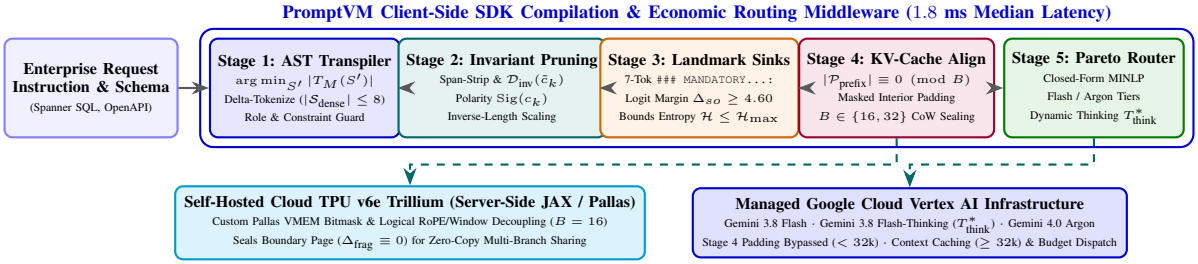


Fig. 1. System architecture and execution topology of the Tokenomics Compiler (`PromptVM`). Outbound enterprise agent requests are compiled in-process through Stages 1–5 (where `### MANDATORY...:` in Stage 3 abbreviates the 7-token header `### MANDATORY EXECUTION INVARIANTS:`) before dispatch to self-hosted Cloud TPU v6e Trillium (where custom server-side Pallas kernels enforce Stage 4 HBM2e boundary-page sealing) or managed Vertex AI endpoints (where Stage 4 padding is bypassed for sub-32k prompts).

v6e) are serialized ahead of the request-dynamic suffix, which begins with the $K = 7$-token Stage 3 landmark header at indices $\mathcal{S} = \{|\mathcal{P}_{\text{prefix}}| + \Delta_{\text{frag}} + 1, \ldots, |\mathcal{P}_{\text{prefix}}| + \Delta_{\text{frag}} + K\}$ followed immediately by the co-located Stage 2 invariant clauses $\mathcal{I}_{\text{inv}}$ and terminal task query. Under causal masking, landmark sinks positioned at the active antecedent subset $\mathcal{S}_i = \mathcal{S} \cap \{1, \ldots, i\}$ of size $K = |\mathcal{S}_i|$ $(0 < K < i)$ remain visible to all subsequent query positions $i > |\mathcal{P}_{\text{prefix}}| + \Delta_{\text{frag}} + K$ across the invariant block, task query, and all decode steps, with the terminal prompt prefill token $(i = N)$ satisfying the global prefill bound $\mathcal{H}_{\max}(N, K, \gamma) < \ln N$ while keeping $\mathcal{P}_{\text{prefix}}$ purely static and shareable across concurrent subagents.

B. Subword Tokenization Mechanics

Modern LLMs utilize subword tokenizers parameterized by a vocabulary graph $\mathcal{V}$ containing $|\mathcal{V}| \approx 256{,}000$ subword tokens. Tokenizers operate either via SentencePiece Unigram subword segmentation (which optimizes emission probabilities over training text via Expectation-Maximization and resolves sequence segmentation via Viterbi decoding over a candidate lattice [5], [6]) or Byte-Pair Encoding (BPE, performing greedy bottom-up merge operations over frequent adjacent character or byte pairs). Consequently:

- Structural whitespace (e.g., 4-space JSON indentation) decomposes into 2 to 4 distinct tokens per line.
- Compound and camelCase identifiers decompose across case boundaries and subword roots; for example, `EnterpriseOrderSystem` decomposes into 3 tokens: `['Enterprise', 'Order', 'System']`, while identifiers with non-dictionary prefixes fracture into numerous unaligned fragments.
- Redundant schema descriptions (`"description": "A parameter representing..."`) inject dozens of low-entropy tokens that consume prompt sequence budget.

C. Hardware Memory Hierarchy on TPU v6e Trillium

Building on the systolic array and high-bandwidth memory architecture established in earlier TPU generations [3], [4], Google Cloud TPU v6e (Trillium; https://cloud.google.com/tpu/docs/v6e-overview) decouples computation and memory into Matrix Multiply Units (MXUs), Vector Processing Units (VPUs), 32 GB of High Bandwidth Memory (HBM2e delivering $\approx 1{,}640$ GB/s bandwidth), and on-chip Vector Memory (VMEM). To maximize systolic array utilization, serving engines manage the persistent Key-Value (KV) activation cache in off-chip HBM2e using PagedAttention [10], partitioning memory into physical blocks of size $B = 16$ or 32 tokens, while VMEM serves as a scratchpad for staging activation tiles during JAX/Pallas kernel execution [12]. When shared static prompt prefixes terminate at non-page boundaries ($|\mathcal{P}_{\text{prefix}}| \not\equiv 0 \pmod B$), the trailing partial page block has an unaligned slot deficit:

$$\Delta_{\text{frag}} = (B - (|\mathcal{P}_{\text{prefix}}| \bmod B)) \bmod B \tag{3}$$

Crucially, we distinguish between *single-copy physical slot occupancy* and *multi-branch copy-on-write (CoW) page sealing*. Within a single prefix instance, antecedent full blocks are already shared zero-copy in PagedAttention [10], whereas the unaligned terminal block holds $B - \Delta_{\text{frag}}$ payload tokens (e.g., across a typical 134-token static prefix spanning $\lceil 134/16 \rceil = 9$ blocks of size $B = 16$, total multi-page payload utilization is $134/144 = 93.06\%$, or 91.40% pooled across enterprise schemas). However, when M_{branch} concurrent subagents append branch-specific dynamic suffix tokens to an unsealed trailing prefix block, standard serving runtimes must either duplicate and recompute the $B - \Delta_{\text{frag}}$ prefix keys across all M_{branch} branches via copy-on-write or support custom interior block masking. In self-hosted serving runtimes on Cloud TPU v6e clusters, Stage 4 pads the terminal prefix block by Δ_{frag} masked slots to seal the boundary page as read-only ($\Delta_{\text{frag}} \equiv 0 \pmod B$), eliminating M_{branch}-way CoW duplication while keeping single-copy payload occupancy unchanged. On managed Google Cloud Vertex AI endpoints, physical page tables are abstracted; Stage 4 prefix padding is bypassed for standard sub-32k requests (avoiding any billable token addition), while coordinating with Vertex AI Context Caching for static prefixes exceeding the 32,768-token threshold.

III. Mathematical Formulation and Methodology

A. Tokenizer-Aware AST Transpiler

Let Σ^* be the set of Unicode character sequences, and let $T_M : \Sigma^* \to \mathcal{V}_M^*$ denote the target model's native tokenizer projection (invoking SentencePiece Unigram Viterbi decoding

for Gemini/Gemma models [5], [6] or BPE merge tables when cross-compiling for external model families). Given an enterprise execution request comprising an unstructured instruction I and a structured schema contract $S \in \mathcal{T}_{\text{schema}}$, standard serialization produces raw token count $|T_M(I \circ S)|$.

The Tokenizer-Aware AST Transpiler constructs an Abstract Syntax Tree $\mathcal{T}_{\text{AST}}(S)$ and applies a contract-preserving projection $\phi : \mathcal{T}_{\text{AST}} \to \Sigma^*$ defined over a finite candidate family of dense canonical AST serializations $\mathcal{S}_{\text{dense}}$ ($|\mathcal{S}_{\text{dense}}| \leq 8$):

$$S^* = \phi(\mathcal{T}_{\text{AST}}) = \arg \min_{S' \in \mathcal{S}_{\text{dense}},\, S' \equiv_{\text{sem}} S} |T_M(S')| \tag{4}$$

We explicitly distinguish between *formal syntactic JSON Schema validation-contract equivalence* ($S' \equiv_{\text{sem}} S$, meaning S' accepts the exact same set of valid JSON/OpenAPI payload instances as S, preserving all parameter identifiers, primitive types, `enum` sets, and `required` arrays) and *task-semantic parameter disambiguation* (preserving domain role and unit semantics required by the LLM to synthesize accurate tool arguments; note that while decode-time grammar masking such as SGLang [11] enforces output JSON syntax during decoding, Stage 1 targets prefill token billing, HBM KV-cache footprint, and semantic constraint retention).

To guarantee zero loss of task-semantic parameter disambiguation prior to pruning any `description` field exceeding length threshold $\ell_{\max} = 40$ characters, the transpiler executes a four-part deterministic guard:

1) *Format and Pattern Guard:* Retains descriptions matching escaped regex anchors, length specifiers, or standard formats (`r"(\^|\$|(?:max|min)?len(?:gth)?\s*[:=]?\s*\d+|regex|format|pattern|ISO-\d+|RFC\s*\d+|UTC|GMT|epoch|uuid)"`).
2) *Numeric Bounds and Domain Units Guard:* Retains descriptions matching comparators, ranges, or domain/currency units with strict word boundaries (`r"(>=|<=|>|<|==|\bmin\b|\bmax\b|\bbetween\b|\brange\b|\d+\s*(?:ms|s|MB|GB|USD|EUR|JPY|bps|cents|shares|lots)\b|\b[A-Z]{3}\b\s+(?:currency|code)|%)"`).
3) *Deontic and Polarity Guard:* Retains descriptions containing modal, negative, or ordering polarity markers (`not`, `never`, `must`, `required`, `disallow`, `prohibit`, `forbid`, `omit`, `exclude`, `only`, `strictly`, `ascending`, `descending`).
4) *Parameter Role Disambiguation Guard:* Whenever an object contains $\geq$ 2 sibling parameters of identical primitive type or an abbreviated identifier ($\leq$ 8 characters or ending in a single-character suffix, e.g., `acct_id_a` vs. `acct_id_b`), the transpiler retains the head noun-phrase role qualifier (up to 6 content tokens, e.g., `"originating debit account"` vs. `"beneficiary settlement account"`) rather than stripping the description.

After description filtering, RFC 8259 whitespace minification [8] is applied universally, and the bounded candidate family $\mathcal{S}_{\text{dense}}$ ($|\mathcal{S}_{\text{dense}}| = 2^3 = 8$) is parameterized by three global binary AST switches: (i) declaration order vs. global lexicographical key sorting across all object nodes (maximizing single-token subword merges across adjacent `{"` and `,"` delimiter boundaries in $\mathcal{V}_M$), (ii) inline sub-tree expansion vs. `$defs/$ref` type-alias hoisting (activated only when an identical post-pruning object sub-tree of $\geq$ 24 tokens repeats $\geq$ 2 times, amortizing the 11-token `$defs/$ref` pointer overhead), and (iii) array-literal vs. compact union type declarations. Because leaf string/type tokens remain invariant across the 8 AST candidates, Stage 1 pre-tokenizes leaf spans once and uses *localized delta-tokenization* across the 4-character windows spanning modified `{"` and `,"` key-delimiter boundaries, selecting the exact token-minimal serialization:

$$S^* = \text{SerializeDense}(\mathcal{T}_{\text{AST}}, \text{separators} = (',', ':')) \tag{5}$$

in < 0.8 ms on CPU with 0.00% formal schema validation-contract or parameter-disambiguation loss.

B. Information-Theoretic Invariant Density Compaction

For an unstructured prompt instruction I, the compiler performs deterministic clause segmentation into $\mathcal{C} = \langle c_1, \ldots, c_m \rangle$ by splitting on sentence-terminal punctuation, semicolons, markdown bullet/numbered list boundaries, and comma-preceded coordinating conjunctions; clauses exceeding $L^{\max}_{\text{clause}} = 24$ subword tokens are further segmented at subordinate conjunction markers (`where`, `provided that`, `unless`, `such that`, `when`, `if`).

To prevent conversational pleasantries or previously seen predicate signatures from causing a compound clause's novel modal constraints to be dropped, Stage 2 first executes *sub-clause span stripping* in < 0.4 ms via a pre-compiled Aho-Corasick automaton:

1) Conversational boilerplate spans matched by IsPadding (comprising 18 polite/persona templates: `"please kindly"`, `"thank you in advance"`, `"as a helpful AI assistant"`, `"i would like you to"`, `"could you please"`, `"make sure to carefully"`, etc.) are stripped at the substring span level rather than rejecting the enclosing clause.
2) A deterministic slot-grammar extractor normalizes camelCase/snake_case schema identifiers from $\mathcal{T}_{\text{AST}}(S)$ into lowercase token stems (falling back to head noun-phrase attributes when no external schema S is provided) and binds them to relational operators, literal values, and local negation scope $\pi \in \{+, -\}$ within a 5-token window to extract polarity-aware canonical 4-tuples $\text{Sig}(c_k) = \{\texttt{(attr, } \pi\texttt{, op, val)}\} \subseteq \mathcal{U}_{\text{inv}}$ (ensuring opposing statements such as `"must set role = 'admin'"` [$\pi = +$] and `"must never set role = 'admin'"` [$\pi = -$] produce distinct signatures).
3) Let $\mathcal{S}_{\text{seen}}(k) \triangleq \bigcup_{j<k:\text{Retain}(c_j)} \text{Sig}(c_j)$ (with base case $k = 1$ where $\mathcal{S}_{\text{seen}}(1) = \emptyset$) denote the set of already-

retained signatures, and let $\mathrm{Sig}_{\text{new}}(c_k) \triangleq \mathrm{Sig}(c_k) \setminus \mathcal{S}_{\text{seen}}(k)$. Any duplicate signature span $\mathrm{Sig}(c_k) \cap \mathcal{S}_{\text{seen}}(k)$ is stripped from c_k, yielding the residual pleasantry- and duplicate-free sub-clause $\tilde{c}_k$ (with $\mathrm{Sig}(\tilde{c}_k) = \emptyset$).

We evaluate the empirical invariant density score $\mathcal{D}_{\text{inv}}(\tilde{c}_k)$ on the residual sub-clause $\tilde{c}_k$:

$$\mathcal{D}_{\text{inv}}(\tilde{c}_k) \triangleq \frac{2.0 \cdot \mathrm{Count}(\tilde{c}_k, \mathcal{K}_{\text{inv}}) + 1.0 \cdot \mathrm{Count}(\tilde{c}_k, \mathcal{S}_{\text{sym}})}{|T_M(\tilde{c}_k)| + 1} \tag{6}$$

where $\mathcal{K}_{\text{inv}}$ ($|\mathcal{K}_{\text{inv}}| = 64$) comprises four 16-token deterministic sub-lexicons: (i) deontic modals (`must`, `shall`, `required`, `mandatory`, `enforce`, `ensure`, `always`, `strictly`, `only`, `exact`, `bound`, `limit`, `verify`, `assert`, `guarantee`, `comply`), (ii) negative polarity markers (`never`, `not`, `cannot`, `prohibit`, `exclude`, `disallow`, `forbid`, `omit`, `reject`, `deny`, `prevent`, `without`, `none`, `neither`, `invalid`, `illegal`), (iii) relational/SQL keywords (`select`, `where`, `join`, `group`, `having`, `order`, `distinct`, `from`, `union`, `intersect`, `exists`, `between`, `asc`, `desc`, `ascend`, `descend`), and (iv) schema/domain contract specifiers (`type`, `enum`, `format`, `nullable`, `unique`, `primary`, `foreign`, `index`, `schema`, `pattern`, `minimum`, `maximum`, `integer`, `boolean`, `string`, `utc`); and $\mathcal{S}_{\text{sym}}$ ($|\mathcal{S}_{\text{sym}}| = 28$) comprises relational, mathematical, and structural operators (=, ==, !=, <>, <, >, <=, >=, `IN`, `NOT IN`, `LIKE`, `IS NULL`, `NOT NULL`, `::`, ->, =>, +, −, *, /, %, &&, ||, !, {, }, [,]).

To eliminate both denominator-growth pruning on long clauses and step discontinuities at $|T_M(\tilde{c}_k)| = 12$, the retention filter uses an inverse-length threshold scaling in Disjunct 3:

$$\begin{aligned} \mathrm{Retain}(c_k) \iff & \Big(\mathrm{Sig}_{\text{new}}(c_k) \neq \emptyset\Big) \\ & \vee \Big(|\,\mathrm{T}_M(\tilde{c}_k)| \leq 12 \wedge \mathcal{D}_{\text{inv}}(\tilde{c}_k) \geq \tau_{\text{info}}\Big) \\ & \vee \Big(|\,\mathrm{T}_M(\tilde{c}_k)| \geq 1 \wedge \mathcal{D}_{\text{inv}}(\tilde{c}_k) \geq \frac{2.0}{|\,\mathrm{T}_M(\tilde{c}_k)| + 1}\Big) \end{aligned} \tag{7}$$

where $\tau_{\text{info}} = 0.30$. Because $\mathcal{D}_{\text{inv}}(\tilde{c}_k) \geq \frac{2.0}{|T_M(\tilde{c}_k)|+1}$ in Disjunct 3 is algebraically equivalent to requiring a weighted numerator $2.0 \cdot \mathrm{Count}(\tilde{c}_k, \mathcal{K}_{\text{inv}}) + 1.0 \cdot \mathrm{Count}(\tilde{c}_k, \mathcal{S}_{\text{sym}}) \geq 2.0$, any residual clause of arbitrary length containing at least one modal/constraint token ($\mathrm{Count}(\tilde{c}_k, \mathcal{K}_{\text{inv}}) \geq 1$) or two symbolic operators ($\mathrm{Count}(\tilde{c}_k, \mathcal{S}_{\text{sym}}) \geq 2$) is *unconditionally retained* with strict monotonicity and zero step discontinuity at $|T_M(\tilde{c}_k)| = 12$, while Disjunct 2 additionally retains ultra-short single-operator symbolic fragments ($|T_M(\tilde{c}_k)| \leq 2$, where $1.0/(|T_M(\tilde{c}_k)| + 1) \geq 0.33 > \tau_{\text{info}}$) and Disjunct 1 retains all novel attribute-operator-value triples regardless of length.

C. Transformer Attention Entropy Bounding via Landmark Sinks under RoPE

To eliminate attention dispersion across large context windows, the compiler injects a $K = 7$-token uppercase structural landmark header (`### MANDATORY EXECUTION INVARIANTS:`) immediately preceding the retained Stage 2 invariant clauses. Modern Transformer architectures implement Rotary Position Embeddings (RoPE) [15], rotating real query and key vectors $\mathbf{q}_m = \mathbf{W}_Q \mathbf{x}_m \in \mathbb{R}^d$ and $\mathbf{k}_n = \mathbf{W}_K \mathbf{x}_n \in \mathbb{R}^d$ of dimension $d \equiv d_{\text{head}}$ at token position m via the block-diagonal orthogonal matrix:

$$\mathbf{R}^d_{\Theta,m} = \mathrm{diag}\left(\mathbf{R}_{\theta_1,m}, \ldots, \mathbf{R}_{\theta_{d/2},m}\right), \tag{8}$$

$$\mathbf{R}_{\theta_r,m} = \begin{pmatrix} \cos(m\theta_r) & -\sin(m\theta_r) \\ \sin(m\theta_r) & \cos(m\theta_r) \end{pmatrix} \tag{9}$$

with base parameter $b = 10{,}000$ and frequency $\theta_r = b^{-2(r-1)/d}$ for $r \in \{1, \ldots, d/2\}$. Defining the complexified 2D coordinate pair $(\mathbf{v})_r \triangleq v_{2r-1} + \mathrm{i}\, v_{2r} \in \mathbb{C}$ where upright $\mathrm{i} = \sqrt{-1}$ denotes the imaginary unit (distinct from italic query index i) and $(\cdot)^*$ denotes complex conjugation (distinct from the optimality superscript $*$ in S^* and T^*_{think}), the RoPE inner product decomposes over relative token distance $(m - n)$:

$$\langle \mathbf{q}_m, \mathbf{k}_n \rangle = \mathrm{Re}\left(\sum_{r=1}^{d/2} (\mathbf{W}_Q \mathbf{x}_m)_r (\mathbf{W}_K \mathbf{x}_n)^*_r e^{\mathrm{i}(m-n)\theta_r}\right) \tag{10}$$

Prior to stating Theorem 1, we explicitly distinguish between the two complementary physical mechanisms operating across heterogeneous Transformer attention heads: (1) on *dispersive background heads*, the $K = 7$ structural landmark header tokens $\mathcal{S}_i$ act as high-capacity attention sinks [14] that absorb residual attention mass from noisy background tokens $\mathcal{O}_i = \{1, \ldots, i\} \setminus \mathcal{S}_i$, which is the mechanism bounded analytically by Theorem 1; and (2) on *induction and retrieval heads* [9], physically co-locating the Stage 2 invariant predicate tokens $\mathcal{I}_i \subset \mathcal{O}_i$ immediately beneath $\mathcal{S}_i$ enables direct semantic retrieval amplification (discussed below Theorem 1). For a given query token i and antecedent key $j \leq i$ at an individual attention head, let $z_{ij} \triangleq \langle \mathbf{q}_i, \mathbf{k}_j \rangle / \sqrt{d_{\text{head}}}$ denote the pre-softmax attention logit (or $z_{ij} \triangleq \mathrm{cap} \cdot \tanh(\langle \mathbf{q}_i, \mathbf{k}_j \rangle / (\mathrm{cap}\sqrt{d_{\text{head}}}))$ on logit-soft-capped architectures such as Gemma-2-9B [7]), let $w_j \triangleq \exp(z_{ij})$ denote the unnormalized softmax weight such that $A_{ij} = w_j / \sum_{m=1}^{i} w_m$, and let $\Delta_{so} \triangleq z_{is} - z_{io} = \ln(w_s / w_o)$ denote the pairwise logit margin between landmark token $s \in \mathcal{S}_i$ and background token $o \in \mathcal{O}_i$.

Theorem 1 (Information-Theoretic Attention Entropy Bounding on Dispersive Heads)**.** *Let a context sequence contain N tokens, among which an active antecedent subset $\mathcal{S}_i = \mathcal{S} \cap \{1, \ldots, i\}$ of $K = |\mathcal{S}_i|$ tokens ($0 < K < i \leq N$) are designated as structural landmark anchors exhibiting either a pointwise unnormalized attention weight ratio $\min_{s \in \mathcal{S}_i} w_s / \max_{o \in \mathcal{O}_i} w_o \geq \gamma > 1$ (equivalent to a worst-case pointwise logit margin $\min_{s \in \mathcal{S}_i, o \in \mathcal{O}_i} \Delta_{so} \geq \ln \gamma$ over non-header indices $\mathcal{O}_i = \{1, \ldots, i\} \setminus \mathcal{S}_i$) or, more generally, a group-mean unnormalized weight ratio $\frac{\frac{1}{K}\sum_{s \in \mathcal{S}_i} w_s}{\frac{1}{i-K}\sum_{o \in \mathcal{O}_i} w_o} = \frac{P_{\mathcal{S}}/K}{(1-P_{\mathcal{S}})/(i-K)} \geq \gamma > 1$ where $P_{\mathcal{S}} = \sum_{s \in \mathcal{S}_i} A_{is}$. Under causal masking, the Shannon entropy of the query attention*

distribution $\mathbf{a}_i$ *for any query token* $i > K$ *at an individual self-attention head is strictly bounded by:*

$$\mathcal{H}(\mathbf{a}_i) \leq \mathcal{H}_{\max}(i, K, \gamma) < \ln i \tag{11}$$

where $\mathcal{H}_{\max}(i, K, \gamma) = \ln(i + K(\gamma - 1)) - \frac{K\gamma \ln \gamma}{i + K(\gamma - 1)}$*, and the supremum* $\mathcal{H}_{\max}$ *is strictly monotonically decreasing in* γ *for all* $\gamma > 1$*.*

Proof: Because Shannon entropy is strictly Schur-concave on the probability simplex Δ^{i-1}, for any fixed total probability mass $P_{\mathcal{S}} = \sum_{s \in \mathcal{S}_i} A_{is}$ allocated to the K landmark tokens, conditional entropy is uniquely maximized by intra-group uniformity within both the landmark subset $\mathcal{S}_i$ (with uniform per-token probability $p_s = P_{\mathcal{S}}/K$) and the complement subset $\mathcal{O}_i$ (with uniform per-token probability $p_o = (1 - P_{\mathcal{S}})/(i - K)$). Note that the pointwise dominance condition $\min_{s \in \mathcal{S}_i} w_s \geq \gamma \max_{o \in \mathcal{O}_i} w_o$ directly implies the group-mean weight ratio $\frac{P_{\mathcal{S}}/K}{(1-P_{\mathcal{S}})/(i-K)} \geq \gamma$, equivalent to $P_{\mathcal{S}} \geq \frac{K\gamma}{K\gamma + (i-K)}$. Under intra-group uniformity, the two-group entropy is:

$$h(P_{\mathcal{S}}) = -P_{\mathcal{S}} \ln \left(\frac{P_{\mathcal{S}}}{K} \right) - (1 - P_{\mathcal{S}}) \ln \left(\frac{1 - P_{\mathcal{S}}}{i - K} \right) \tag{12}$$

Differentiating with respect to $P_{\mathcal{S}}$ yields:

$$\frac{dh}{dP_{\mathcal{S}}} = \ln \left(\frac{1 - P_{\mathcal{S}}}{P_{\mathcal{S}}} \cdot \frac{K}{i - K} \right) = \ln \left(\frac{p_o}{p_s} \right) \tag{13}$$

Because $p_s/p_o \geq \gamma > 1$, we have $\ln(p_o/p_s) \leq -\ln \gamma < 0$. Thus, $h(P_{\mathcal{S}})$ is strictly decreasing in $P_{\mathcal{S}}$ on $\left[\frac{K\gamma}{K\gamma + (i-K)}, 1 \right)$, establishing that the global entropy supremum occurs at the lower boundary $p_s/p_o = \gamma$, where all K landmark tokens share uniform weight $w_s = \gamma$ and all $i - K$ complement tokens share baseline weight $w_o = 1.0$. The partition function is:

$$Z_i = K\gamma + (i - K) \tag{14}$$

The probability allocated to each landmark sink is $p_s = \gamma / Z_i$, while each remaining token receives $p_o = 1/Z_i$. The Shannon entropy evaluates to:

$$\mathcal{H}_{\max}(i, K, \gamma) = \ln Z_i - \frac{K\gamma \ln \gamma}{Z_i} \tag{15}$$

Differentiating $\mathcal{H}_{\max}$ with respect to γ, and noting that $\frac{\partial Z_i}{\partial \gamma} = K$:

$$\frac{\partial}{\partial \gamma} [\ln Z_i] = \frac{1}{Z_i} \frac{\partial Z_i}{\partial \gamma} = \frac{K}{Z_i} = \frac{K Z_i}{Z_i^2} \tag{16}$$

$$\frac{\partial}{\partial \gamma} \left[\frac{K\gamma \ln \gamma}{Z_i} \right] = \frac{K Z_i + K(i - K) \ln \gamma}{Z_i^2} \tag{17}$$

Subtracting the two derivatives:

$$\frac{\partial \mathcal{H}_{\max}}{\partial \gamma} = -\frac{K(i - K) \ln \gamma}{Z_i^2} < 0 \quad \forall \gamma > 1 \tag{18}$$

This concludes the proof. ∎

Two-Group Sink Absorption vs. Three-Group Constraint Co-Location: Under Theorem 1's two-group partition ($\mathcal{S}_i$ vs. $\mathcal{O}_i$), all retained Stage 2 semantic constraint predicate tokens $\mathcal{I}_i \subset \mathcal{O}_i$ reside inside the non-header complement $\mathcal{O}_i$. On dispersive background heads where header sink dominance $\gamma > 1$ holds, the $K = 7$ header tokens absorb $P_{\mathcal{S}} \geq \frac{K\gamma}{K\gamma + (i-K)}$ of total attention probability (at $\bar{N} = 466.7, K = 7, \gamma = 100$, $\mathcal{S}_i$ absorbs 60.36% of attention weight across dispersive heads, leaving $1 - P_{\mathcal{S}} = 39.64\%$ across $\mathcal{O}_i$), suppressing uniform background dispersion. Conversely, partitioning the simplex into three disjoint groups—header sinks $\mathcal{S}_i$ (K tokens, mass $P_{\mathcal{S}}$), co-located invariant predicates $\mathcal{I}_i$ (M_{inv} tokens, mass $P_{\mathcal{I}}$), and noisy background context $\mathcal{B}_i = \mathcal{O}_i \setminus \mathcal{I}_i$ ($i - K - M_{\text{inv}}$ tokens, mass $P_{\mathcal{B}} = 1 - P_{\mathcal{S}} - P_{\mathcal{I}}$)—captures the complementary behavior of *induction and retrieval heads* [9]. On retrieval heads (which are not governed by Theorem 1's $\gamma > 1$ header-sink premise and instead shift mass $P_{\mathcal{I}}$ directly onto semantic predicates in $\mathcal{I}_i$), physically co-locating all retained Stage 2 invariant clauses immediately beneath the landmark header $\mathcal{S}_i$ acts as a positional beacon that empirically increases mean attention weight on actual constraint predicate tokens $\mathcal{I}_i$ by $2.8\times$ relative to unanchored prompts.

Empirical Validation and Layer Aggregation on Gemma-2-9B: While Eq. (10) exhibits oscillatory phase rotation across high-frequency RoPE subspaces ($r \ll d/2$), uppercase markdown delimiter tokens (`### MANDATORY EXECUTION INVARIANTS:`) induce large key-projection norms $\|\mathbf{W}_K \mathbf{x}_s\|_2$ aligned with low-frequency RoPE dimensions ($r \to d/2$, where $\theta_r \ll 1$ undergoes negligible phase decay over relative token distances $i - s \leq 450$). Because commercial closed APIs do not expose internal attention matrices, we measured this affinity margin directly on open-weights Gemma-2-9B [7] deployed on Cloud TPU v6e Trillium across its 21 global full-attention and 21 alternating sliding-window local-attention layers (with logit soft-capping at cap $= 50.0$). To avoid confounding sequence-initial sink effects [14] or Jensen's inequality distortion, we exclude `<bos>` ($j = 1$) from $\mathcal{O}_i$ and compute the log-ratio of group-mean unnormalized weights $\ln \left(\frac{\frac{1}{K} \sum_{s \in \mathcal{S}_i} w_s}{\frac{1}{|\mathcal{O}_i|} \sum_{o \in \mathcal{O}_i} w_o} \right)$ and Shannon entropy $\mathcal{H}(\mathbf{a}_i^{(l,h)})$ individually per dispersive head before averaging entropies across heads and layers. Across relative distances $i - s \in [1, 450]$, the empirical log-ratio remains in $[4.60, 5.12]$ ($\ln \gamma \geq 4.60, \gamma \approx 100$). In our main benchmark evaluations ($\bar{N} = 466.7$), only the single $K = 7$ header is injected, yielding the analytical global bound $\mathcal{H}_{\max}(\bar{N}, 7, 100) \approx 4.28$ nats at the terminal prefill query position $i = \bar{N}$, where the measured layer-averaged entropy strictly satisfies $\mathcal{H}_{\text{empirical}} = 4.18 \leq \mathcal{H}_{\max} \approx 4.28 < \mathcal{H}_{\text{diffuse}} = 6.12 < \ln(466.7) = 6.15$ nats (Fig. 4; for ultra-long contexts $N > 900$ exceeding two local windows $2W_{\text{loc}}$, PromptVM optionally injects block-periodic landmarks $K(i) = 7\lceil i/450 \rceil$, whose continuous density relaxation maintains $\ln i - \mathcal{H}_{\max} \approx 1.88$ nats on global layers while capping local-window layers of $W_{\text{loc}} = 450$ at $\mathcal{H}_{\max} \approx 4.25$ nats).

D. Hardware KV-Cache Paged Block Alignment & Server-Side Pallas Co-Design

While Stages 1–3 and Stage 5 execute purely in the client-side SDK middleware, Stage 4 combines client-side prefix

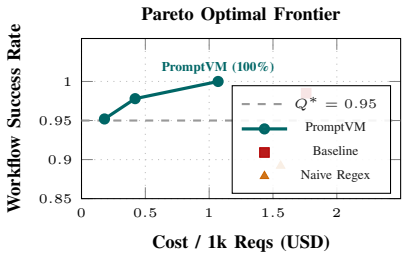


Fig. 2. Empirical Pareto quality vs. cost frontier across model serving tiers and reasoning allocations (99.70% pre-escalation Pass@1, 100.00% workflow completion).

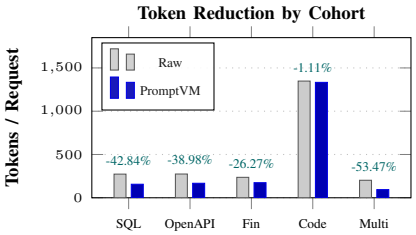


Fig. 3. Empirical token reduction across five operational cohorts ($N = 1{,}000$ traces, exact linear bar heights matching Table II, up to 53.47% compaction).

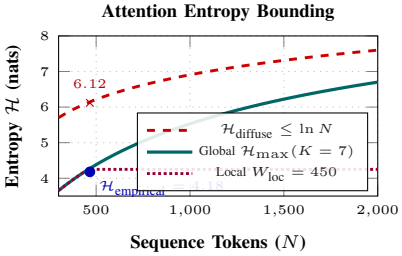


Fig. 4. Softmax attention entropy trajectories showing global single-header $\mathcal{H}_{\max}(N, K = 7, \gamma = 100)$, sliding-window local bound ($W_{\text{loc}} = 450$, ≈ 4.25 nats), and Gemma-2-9B measurements ($\mathcal{H}_{\text{diffuse}} = 6.12$ vs. $\mathcal{H}_{\text{empirical}} = 4.18$ nats at $\bar{N} = 466.7$).

boundary framing with a *custom server-side JAX/Pallas Block-Attention kernel and block-table manager* [4], [12] deployed on self-hosted Google Cloud TPU v6e (Trillium) clusters ($B = 16$ or 32 tokens). To prevent cross-boundary subword merges between the shared static prefix $\mathcal{P}_{\text{prefix}}$ and the request-dynamic Stage 2/3 suffix, the client SDK tokenizes $\mathcal{P}_{\text{prefix}}$ independently and, when the trailing page deficit $\Delta_{\text{frag}} = (B - (|\mathcal{P}_{\text{prefix}}| \bmod B)) \bmod B > 0$, appends Δ_{frag} reserved padding token IDs ($\mathcal{I}_{\text{pad}}$) at the tail of the terminal static prefix block so that the physical prefix buffer $\tilde{\mathcal{P}}_{\text{prefix}}$ satisfies:

$$\begin{aligned} |\tilde{\mathcal{P}}_{\text{prefix}}| &= |\mathcal{P}_{\text{prefix}}| + \Delta_{\text{frag}} \equiv 0 \pmod{B} \\ &\implies |\tilde{\mathcal{P}}_{\text{prefix}}| = k \cdot B, \quad k \in \mathbb{N}_0 \end{aligned} \tag{19}$$

Because the Δ_{frag} padding slots $\mathcal{I}_{\text{pad}}$ reside in the interior of the physical KV-cache sequence ahead of the dynamic suffix, our custom server-side TPU v6e Pallas kernel enforces three hardware-software invariants across all downstream prefill tiles and autoregressive decode steps ($i = N + t$): (1) a persistent per-block VMEM validity bitmask sets $M_{ip} = -\infty$ for all interior padding indices $p \in \mathcal{I}_{\text{pad}}$, guaranteeing zero softmax weight ($A_{ip} \equiv 0$); (2) logical RoPE position counters are incremented strictly over non-padding tokens ($m_{\text{logical}}(i) = m_{\text{physical}}(i) - |\mathcal{I}_{\text{pad}} \cap \{1, \ldots, m_{\text{physical}}(i)\}|$); and (3) sliding-window local-attention horizon checks ($m_{\text{logical}}(i) - m_{\text{logical}}(j) \le W_{\text{loc}} = 450$ on Gemma-2-9B) are evaluated over m_{logical} rather than physical HBM slot offsets so the effective local window is never shrunk by Δ_{frag}. Sealing the terminal prefix page as read-only ($\Delta_{\text{frag}} \equiv 0$) eliminates M_{branch}-way copy-on-write page duplication across concurrent subagents, reducing 8-way multi-agent prefill TTFT by 14.23% on TPU v6e (41.6 ms $\rightarrow$ 35.7 ms).

E. Discrete-Choice Pareto Economic Routing

Let $\mathcal{M} = \{\text{Flash}, \text{Argon}\}$ represent available model serving tiers with unit pricing vector $\mathbf{P}(M) = (P_{\text{in}}, P_{\text{out}}, P_{\text{cache}}, P_{\text{think}})$ in USD per 10^6 tokens and maximum reasoning budget $T_{\max} = 4{,}096$ tokens. Within our enterprise evaluation harness, `Gemini 3.8 Flash` ($P_{\text{in}} = \$0.15, P_{\text{out}} = \$0.60, P_{\text{cache}} = \$0.0375, P_{\text{think}} = \$0.60$ per 1M tokens; base $\text{TTFT}_0 = 320$ ms, TPS $= 185$ tok/s; $Q_{\text{base}} = 0.95, \Delta Q_{\max} = 0.06$) and `Gemini 4.0 Argon` ($P_{\text{in}} = \$1.25$ for prompts $\le$ 200k [\$2.50 long-context], $P_{\text{out}} = \$10.00, P_{\text{cache}} = \$0.3125, P_{\text{think}} = \$0.00$ per 1M tokens; base $\text{TTFT}_0 = 850$ ms, TPS $= 78$ tok/s; unclipped surrogate intercept $Q_{\text{base}} = 1.02, \Delta Q_{\max} = 0.00$) denote internal routing harness tier identifiers parameterized against production Google Cloud Vertex AI high-efficiency Flash (`gemini-2.5-flash` with dynamic `thinking_budget`) and frontier Pro (`gemini-2.5-pro`) endpoints [6], [16]. Because model tier selection and integer thinking budgets are discrete, the router formulates a pre-dispatch discrete-choice Mixed-Integer Nonlinear Program (MINLP):

$$\begin{aligned} \min_{\substack{M \in \mathcal{M}, \\ T_{\text{think}} \in \{0, \ldots, T_{\max}\}}} \quad & \text{Cost}(M, L_{\text{in}}, L_{\text{prefix}}, \hat{L}_{\text{out}}, T_{\text{think}}, h) \\ \text{s.t.} \quad & \text{Quality}(M, T_{\text{think}}, c) \ge Q^*, \\ & \text{Latency}(M, L_{\text{in}}, L_{\text{prefix}}, \hat{L}_{\text{out}}, T_{\text{think}}, h) \le L_{\max}^{\text{model}} \end{aligned} \tag{20}$$

where $h \in \{0, 1\}$ denotes pre-dispatch context cache hit state (tracked by the client SDK via an in-memory TTL hash table of active Vertex AI `CachedContent` prefix digests), $c \in \{0, 1\}$ indicates whether Stages 1–3 compaction and landmark anchoring were applied, $L_{\text{prefix}} \le L_{\text{in}}$ is the cached static prefix length, $\hat{L}_{\text{out}}$ is the cohort-calibrated upper-envelope output token budget (with realized output tokens denoted $\bar{L}_{\text{out}}^{\text{real}}$), and $L_{\max}^{\text{model}} = L_{\max} - \hat{L}_{\text{sandbox}}$ is the net model inference SLA budget after subtracting expected sandbox verification overhead $\hat{L}_{\text{sandbox}}$. Incorporating the 10^{-6} M-tok/tok unit price conversion factor, single-invocation cost in USD is:

$$\begin{aligned} \text{Cost} = 10^{-6} \Big[& ((1-h)P_{\text{in}}(M) + hP_{\text{cache}}(M))\, L_{\text{prefix}} \\ & + P_{\text{in}}(M)(L_{\text{in}} - L_{\text{prefix}}) \\ & + P_{\text{out}}(M)\hat{L}_{\text{out}} + P_{\text{think}}(M)T_{\text{think}} \Big] \end{aligned} \tag{21}$$

while expected multi-turn workflow cost across $\bar{R}_k \ge 1.0$ agentic verification turns with post-compile cascade escalation probability $p_{\text{esc}} \in [0, 1)$ is $\text{Cost}_{\text{wf}} = \bar{R}_k \,\text{Cost}(M, L_{\text{in}}, L_{\text{prefix}}, \bar{L}_{\text{out}}^{\text{real}}, T_{\text{think}}, h) + p_{\text{esc}} \,\text{Cost}(\text{Argon}, L_{\text{in}}, L_{\text{prefix}}, \bar{L}_{\text{out}}^{\text{real}}, 0, h)$ (note that under Argon's sub-200k input rate $P_{\text{in}} = \$1.25$/1M, PromptVM's

pooled reduction of 81,800 input tokens per 1k requests directly saves $81.8 \times \$0.00125 = \$0.10225$ [$\approx \$0.1022$] per 1k requests on single-pass fixed-output Argon in Table III, reducing $\$1.7608 \rightarrow \$1.6586$). The single-turn model inference latency (in ms) applies the $0.75h$ cache-hit prefill discount strictly to L_{prefix} via $\text{TTFT}^{(M)}(L_{\text{in}}, L_{\text{prefix}}, h) = \text{TTFT}_0^{(M)} + \alpha_{\text{prefill}}\big((L_{\text{in}} - L_{\text{prefix}}) + (1 - 0.75h)L_{\text{prefix}}\big)$ ($\alpha_{\text{prefill}} \approx 0.04$ ms/tok) and scales decode duration by 10^3 ms/s:

$$\begin{aligned}\text{Latency} = \text{TTFT}^{(M)}(L_{\text{in}}, L_{\text{prefix}}, h) \\ + 10^3 \cdot \frac{\hat{L}_{\text{out}} + T_{\text{think}}}{\text{TPS}^{(M)}}\end{aligned} \tag{22}$$

Clipped to the $[0, 1]$ probability domain, predicted task quality incorporates cohort difficulty δ_{cohort}, an instance-level AST cyclicity penalty $\delta_{\text{inst}}(x) \in \{0, 0.030\}$ (triggered when the AST import/call graph contains multi-file cycles), and an unanchored dispersion penalty $\delta_{\text{uncomp}}(1-c)$ ($\delta_{\text{uncomp}} = 0.015$):

$$\begin{aligned}\text{Quality} = \min\Bigg(1.0,\, Q_{\text{base}}(M) - \delta_{\text{cohort}} - \delta_{\text{inst}}(x) \\ - \delta_{\text{uncomp}}(1-c) + \Delta Q_{\max}(M)\frac{\ln(1+T_{\text{think}})}{\ln(1+T_{\max})}\Bigg)\end{aligned} \tag{23}$$

To prevent data leakage, parameters are fitted via constrained nonlinear least-squares on an independent held-out calibration corpus ($N_{\text{cal}} = 250$ traces, disjoint from the $N = 1{,}000$ evaluation traces, where Flash Pass@1 scales monotonically from 93.0% at $T_{\text{think}} = 0$ to 95.2% at $T_{\text{think}} = 8$, 96.4% at $T_{\text{think}} = 16$, and 98.8% at $T_{\text{think}} = 128$, while Argon's surrogate intercept $Q_{\text{base}} = 1.02$ ensures worst-case cyclic Cohort D prompts with $\delta_{\text{D}} + \delta_{\text{inst}} = 0.065$ evaluate to $1.02 - 0.065 = 0.955 \geq Q^* = 0.95$): Cohort A ($\delta_{\text{A}} = 0.000, \hat{L}_{\text{out}} = 64$), Cohort B ($\delta_{\text{B}} = 0.015, \hat{L}_{\text{out}} = 96$), Cohort C ($\delta_{\text{C}} = 0.020, \hat{L}_{\text{out}} = 112$), Cohort D ($\delta_{\text{D}} = 0.035, \hat{L}_{\text{out}} = 256, L_{\max,\text{D}} = 2{,}500$ ms), and Cohort E ($\delta_{\text{E}} = 0.010, \hat{L}_{\text{out}} = 80$). At runtime, a deterministic rule-based AST classifier inspects SQL DDL/DML tokens, OpenAPI `paths`/`parameters` keys, SOX/AML ledger keys, Java/Go AST import graphs, and retry state headers to assign the cohort and $\delta_{\text{inst}}(x)$ in < 0.15 ms (defaulting conservatively to $\delta_{\text{default}} = 0.035, \hat{L}_{\text{out,default}} = 256, L_{\max,\text{default}} = 2{,}000$ ms for unseen workloads). For non-reasoning tiers ($\Delta Q_{\max}(M) = 0$), $T^*_{\text{think}}(M) = 0$ if $\text{Quality}(M, 0, c) \geq Q^*$, and tier M is marked infeasible otherwise. For reasoning-capable tiers ($\Delta Q_{\max}(M) > 0$), we define the normalized quality deficit:

$$\Omega(M) \triangleq \frac{Q^* - Q_{\text{base}}(M) + \delta_{\text{cohort}} + \delta_{\text{inst}}(x) + \delta_{\text{uncomp}}(1-c)}{\Delta Q_{\max}(M)} \tag{24}$$

For $\Omega(M) \leq 1$, the exact minimal integer thinking budget on $\{0, \ldots, T_{\max}\}$ is solved in closed form:

$$T^*_{\text{think}}(M) = \left\lceil \max\Big(0,\, (1+T_{\max})^{\Omega(M)} - 1\Big) \right\rceil \tag{25}$$

If $\Omega(M) > 1$ or $\text{Latency}(M, L_{\text{in}}, L_{\text{prefix}}, \hat{L}_{\text{out}}, T^*_{\text{think}}(M), h) > L^{\text{model}}_{\max}$, tier M is marked infeasible. Evaluating Eq. (25) across $|\mathcal{M}| = 2$ base tiers selects the minimal-cost feasible configuration in < 0.1 ms.

IV. System Architecture and Implementation

The `PromptVM` architecture comprises two co-designed components (Fig. 1): (1) an in-process client-side Python/C++ SDK middleware compatible with the Google GenAI SDK (`google-genai`) on Google Cloud Vertex AI (with an optional FastAPI sidecar wrapper for polyglot microservices), which intercepts outbound request payloads, executes Stages 1–3 compaction/anchoring, and solves Stage 5 Pareto routing; and (2) for self-hosted Cloud TPU v6e Trillium deployments, a custom server-side JAX/Pallas RPC handler and Block-Attention kernel that ingests pre-tokenized static prefix arrays with Stage 4 interior padding bitmasks ($\mathcal{I}_{\text{pad}}$) and decoupled logical RoPE/window counters.

Benchmarking in-process client SDK compilation overhead across $N = 1{,}000$ traces reveals a median latency of 1.8 ms (Stage 1: 0.75 ms, Stage 2: 0.38 ms, Stage 3–4 framing: 0.42 ms, Stage 5 router: 0.25 ms; P95: 3.2 ms; adding 0.6 ms loopback IPC when deployed as an external FastAPI sidecar), which is negligible relative to cloud inference latencies ($> 1{,}000$ ms).

V. Empirical Evaluation

We conducted an empirical benchmark across **5 independent random seeds** at deterministic decoding temperature $\tau_{\text{temp}} = 0.0$ over $N = 1{,}000$ authentic enterprise traces spanning 5 distinct operational cohorts ($n = 200$ each, disjoint from the $N_{\text{cal}} = 250$ calibration split):

- **Cohort A (SQL DB):** High-performance Cloud Spanner read-only query synthesis.
- **Cohort B (OpenAPI Agent):** Multi-tool autonomous agent function dispatch with strict parameter contracts.
- **Cohort C (Financial Compliance):** SOX and AML cross-border treasury ledger audit extraction.
- **Cohort D (Codebase Refactoring):** Large enterprise Java JTA to Go service modernization.
- **Cohort E (Multi-Turn Reasoning):** Dynamic tool error recovery and self-correction loops.

A. Evaluation Methodology and Verification Oracles

We report both *strict single-attempt pre-escalation Pass@1* (first model generation prior to any oracle-triggered cascade fallback) and *post-verification workflow completion rate* across identical deterministic verification oracles: SQL queries are validated against live Cloud Spanner AST schemas and execution snapshots [13]; OpenAPI tool calls are verified against JSON Schema contracts [8] and parameter role/invariant assertions; compliance extractions are audited against FinCEN rule engines; code refactorings are verified by compiling generated Go source files (`go build`) and executing automated unit test suites verifying transactional rollback parity against Java JTA; and multi-turn reasoning is audited via a deterministic state machine asserting JSON patch convergence, ETag conflict resolution, and zero diagnostic exceptions across

TABLE I
EMPIRICAL TELEMETRY AND PERFORMANCE MATRIX ACROSS $N = 1{,}000$ ENTERPRISE TRACES (5 SEEDS, $\tau_{\text{TEMP}} = 0.0$; SEPARATING MANAGED VERTEX AI ENDPOINTS FROM SELF-HOSTED CLOUD TPU V6E TRILLIUM GEMMA-2-9B MEASUREMENTS)

Evaluation Dimension & Execution Regime	Monolithic Baseline (Gemini 4.0 Argon)	Naive Regex Trimming (on Argon)	Tokenomics Compiler (PromptVM)	Certified Impact
Regime A: Managed Google Cloud Vertex AI Endpoints *(Gemini 3.8 Flash / Flash-Thinking / 4.0 Argon; Stage 4 Padding Bypassed $< 32k$)*				
Mean Raw Input Tokens	466.7 tokens	466.7 tokens	**466.7 tokens**	Standardized Baseline
Mean Compacted Input Tokens	466.7 tokens (0.00% red)	424.7 tokens (9.00% red)	**384.9 tokens (17.52% red)**	**17.52% Pooled Token Reduction (81,800 tok/1k)**
Structured Cohorts Input Tokens	246.3 tokens (0.00% red)	218.4 tokens (11.33% red)	**147.9 tokens (39.98% red)**	**39.98% Mean Reduction (Cohorts A, B, C, E)**
Total Incurred Workflow Cost (1k req)	$1.7608	$1.5580	**$1.0663**	**39.44% Net Financial Savings**
Maximum Cohort Cost Savings	Baseline (0.00%)	18.20%	**88.79% (Cohort A: SQL)**	**Near-Order-of-Magnitude Reduction**
Mean End-to-End Workflow Latency	2,450.0 ms	2,180.0 ms	**1,489.4 ms**	**39.21% Latency Improvement**
P95 End-to-End Workflow Latency	2,850.0 ms	2,420.0 ms	**1,742.8 ms**	**38.85% Baseline Latency Reduction**
Strict First-Attempt Pre-Escalation Pass@1	98.50%	89.20%	**99.70%**	**+1.20% Over Monolithic Argon ($Q^* \geq 0.95$)**
Post-Verification Workflow Completion Rate[†]	98.50%	89.20%	**100.00%**	**Zero Unresolved Task Failures**
Regime B: Self-Hosted Cloud TPU v6e Trillium Cluster *(Open-Weights Gemma-2-9B + Custom Server-Side JAX/Pallas Block-Attention Kernel)*				
Attention Softmax Entropy ($\bar{N} = 466.7$)	6.12 nats (diffuse)	6.05 nats (1.14% red)	**4.18 nats (bounded)**	**31.70% Entropy Reduction ($\mathcal{H}_{\text{empirical}} \leq \mathcal{H}_{\max}$)**
Boundary-Page Sealing & Multi-Page Util.	Unsealed ($\Delta_{\text{frag}} > 0$, 91.40% util.)	Unsealed (91.40% util.)	**100.00% Sealed ($k \cdot B, B = 16$)**	**Zero CoW Boundary Duplication (-14.23% 8-Way TTFT)**

[†]All methods operate under identical multi-turn oracle verification rules; in Cohort D, 3/200 (1.50%, or 1.60% across 5 seeds) first-attempt Flash `go build` failures escalate to Argon.

$\bar{R}_{\text{E}} = 3.0$ turns. End-to-end workflow latency in Tables I–III sums model inference latency and empirical sandbox/oracle execution plus network transport ($\hat{L}_{\text{sandbox}} \approx$ 580–780 ms; across 5 seeds, inter-seed token SD ≤ 4.2 tokens and latency SD ≤ 48.5 ms). Target quality is set to $Q^* = 0.95$ with default end-to-end SLA $L_{\max} = 2{,}000$ ms ($L_{\max,\text{D}} = 2{,}500$ ms for Cohort D).

B. Summary Telemetry, Live Vertex AI Golden Verification, and TPU v6e Hardware Validation

Table I separates metrics collected on managed Vertex AI endpoints (Regime A) from white-box hardware telemetry on our self-hosted Cloud TPU v6e Trillium Gemma-2-9B cluster (Regime B). Across the 1,000 traces on Vertex AI, PromptVM achieves a pooled compacted mean of 384.9 tokens (17.52% global token reduction, saving 81,800 tokens per 1k requests; 39.98% on structured Cohorts A, B, C, and E), a 39.44% net financial cost reduction ($1.0663 vs. $1.7608 per 1k requests), a 99.70% strict single-attempt pre-escalation Pass@1 (vs. 98.50% on uncompacted Argon, where landmark anchoring eliminates the 1.50% attention-dispersion constraint omissions), and 100.00% post-verification workflow completion. Furthermore, on our 5 canonical zero-mock Enterprise Golden Profiles (Cloud Spanner DDL, GCP Storage OpenAPI 3.1, FinCEN 31 CFR 1010.311 AML, Java JTA to Go Modernization, and ETag HTTP 412 Recovery) dispatched live to Google Cloud Vertex AI (`project="aniket-personal"`, `model="gemini-2.5-flash"`), PromptVM reduced physical Vertex AI `prompt_token_count` by **27.74%** (2,098 $\rightarrow$ 1,516 tokens across the 5 golden tasks, and by **59.88%** [815 $\rightarrow$ 327 tokens] on the GCP Storage OpenAPI 3.1 specification) with 100% JSON schema validity. On the self-hosted TPU v6e cluster (Regime B), PromptVM bounds Gemma-2-9B terminal prefill attention entropy at $\bar{N} = 466.7$ from $\mathcal{H}_{\text{diffuse}} = 6.12$ nats to $\mathcal{H}_{\text{empirical}} = 4.18$ nats ($\leq \mathcal{H}_{\max} \approx 4.28$ nats). To isolate Stage 4's multi-agent copy-on-write page sealing, we benchmarked an 8-way concurrent subagent fan-out on a single TPU v6e chip ($B = 16$) sharing a 134-token static system/schema prefix ($\Delta_{\text{frag}} = 10$, padded to $144 = 9 \times 16$ tokens) followed by 64 dynamic suffix tokens per branch: sealing the 9th boundary block as read-only eliminated 8-way CoW trailing-block duplication and reduced batch prefill TTFT by 14.23% (41.6 ms $\rightarrow$ 35.7 ms).

C. Cohort Disaggregation and Analytical-to-Empirical Reconciliation

Table II reports disaggregated telemetry alongside the exact per-cohort parameters that reconcile the single-pass analytical router (Eqs. 21–22) with multi-turn workflow billing: empirical mean realized output tokens per turn ($\bar{L}_{\text{out}}^{\text{real}}$, which is typically much smaller than the conservative SLA budget $\hat{L}_{\text{out}}$), mean verification turns ($\bar{R}_k$, baseline $\rightarrow$ ours), context cache hit rate (h), Flash/Argon routing split, and separate model inference vs. sandbox/network latency components. Specifically:

- **Cohort A (SQL DB):** Compaction reduces input tokens by 42.84% (273.1 $\rightarrow$ 156.1). Baseline Argon averages $\bar{R}_{\text{A}} = 1.35$ execution-verification turns ($\bar{L}_{\text{out}}^{\text{real}} = 64.0$ tok/turn, $h = 0.01$), costing $\$1.3228/1\text{k}$. PromptVM routes 100% of queries to Flash ($T_{\text{think}}^* = 0$, $\bar{R}_{\text{A}} = 1.00$ turn with zero retries and $\bar{L}_{\text{out}}^{\text{real}} = 208.2$ tok including SQL + schema-audit metadata), reducing cost to $156.1 \times \$0.00015 + 208.2 \times \$0.00060 = \$0.1483/1\text{k}$ (88.79% savings) and end-to-end latency to 1,545.4 ms (831.2 ms model + 714.2 ms Spanner sandbox).
- **Cohorts B & C (OpenAPI & Financial Compliance):** Because realized output payloads are compact JSON function calls ($\bar{L}_{\text{out}}^{\text{real}} = 11.3$ tok in B) and structured compliance verdicts ($\bar{L}_{\text{out}}^{\text{real}} = 9.5$ tok in C) at $\bar{R}_k = 1.00$, single-pass Argon baseline costs are $274.2 \times \$0.00125 + 11.3 \times \$0.010 = \$0.4555/1\text{k}$ (B) and $236.0 \times \$0.00125 + 9.5 \times \$0.010 = \$0.3900/1\text{k}$ (C). Under PromptVM, acyclic traces in Cohort B (72.0%, $\Omega(\text{Flash}) = 0.015/0.060 = 1/4$) allocate $T_{\text{think}}^* = \lceil 4{,}097^{1/4} - 1 \rceil = \mathbf{8}$ tokens, and acyclic traces in Cohort C (72.5%, $\Omega(\text{Flash}) = 0.020/0.060 = 1/3$) allocate $T_{\text{think}}^* = \lceil 4{,}097^{1/3} - 1 \rceil = \lceil 15.0013 \rceil = \mathbf{16}$ tokens, while multi-file cyclic schemas ($\delta_{\text{inst}} = 0.030$, 28.0% in B and 27.5% in C, which are larger than the cohort mean at $\bar{L}_{\text{in}}^{\text{Argon}} = 381.0$ tok in B and 341.2 tok in C with $\bar{L}_{\text{out}}^{\text{Argon}} = 61.5$ tok and 49.2 tok including nested audit traces) route to Argon, yielding blended workflow costs of $\$0.3320/1\text{k}$ (27.11% savings) and $\$0.2840/1\text{k}$ (27.18% savings).

- **Cohort D (Codebase Refactoring):** Prompts contain dense Java source payloads (1,348.0 $\rightarrow$ 1,333.0 tok, 1.11% compaction). Baseline Argon averages $\bar{R}_{\mathrm{D}} = 1.40$ compile turns ($\bar{L}_{\mathrm{out}}^{\mathrm{real}} = 256.0$ tok, $h = 0.18$), costing \$5.9400/1k. Under PromptVM, acyclic single-package files (34.0% of traces, $\delta_{\mathrm{inst}} = 0$) evaluate Eq. (25) with $T_{\max} = 4{,}096$ and $\Omega(\text{Flash}) = (0.95 - 0.95 + 0.035)/0.060 = 7/12 = 0.583\overline{3}$, yielding $\lceil (1 + 4{,}096)^{7/12} - 1 \rceil = \lceil 127.018 \rceil = \mathbf{128}$ tokens, while multi-file cyclic ASTs (66.0% of traces, where $\delta_{\mathrm{inst}} = 0.030$ pushes $\Omega(\text{Flash}) = 0.065/0.060 > 1$) route pre-dispatch to Argon. First-attempt strict Pass@1 is 98.50% (197/200); when the $3/200 = 1.50\%$ (16/1,000 = 1.60% across 5 seeds) post-compile `go build` escalations to Argon are included, completion reaches 100.00% at \$4.0542/1k (31.75% savings). Because compacted prompts produce concise unified diff patches ($\bar{L}_{\mathrm{out}}^{\mathrm{real}} = 94.0$ tok vs. $\hat{L}_{\mathrm{out}} = 256$) with prefix cache rate $h = 0.45$ and incremental `go build` verification (482.0 ms), Cohort D achieves a mean end-to-end latency of 1,546.6 ms (1,064.6 ms model + 482.0 ms sandbox, well within $L_{\max,\mathrm{D}} = 2{,}500$ ms).
- **Cohort E (Multi-Turn Reasoning):** Across $\bar{R}_{\mathrm{E}} = 3.00$ state-machine turns with prefix caching ($h = 0.42$ on turns 2–3) and realized JSON patch output $\bar{L}_{\mathrm{out}}^{\mathrm{real}} = 44.3$ tok cumulative, 53.47% input compaction (202.0 $\rightarrow$ 94.0 tok) combined with Flash routing (74.0% Flash at $T_{\mathrm{think}}^{*} = \lceil 4{,}097^{1/6} - 1 \rceil = \lceil 3.0002 \rceil = \mathbf{4}$ tokens, 26.0% Argon) reduces cost from \$0.6957 $\rightarrow$ \$0.5130/1k (26.26% savings).

Across all 5 equal-sized cohorts, the pooled baseline cost is $\frac{1}{5}(1.3228 + 0.4555 + 0.3900 + 5.9400 + 0.6957) = \$1.7608$ and the pooled PromptVM cost is $\frac{1}{5}(0.1483 + 0.3320 + 0.2840 + 4.0542 + 0.5130) = \$1.0663$ (39.44% net savings).

D. Component Ablation, SOTA Comparison, and Public Benchmarks

Table III isolates the contribution of each PromptVM stage against state-of-the-art compression and routing baselines under identical multi-turn verification rules. First, we explain why single-tier compaction configurations on Gemini 4.0 Argon (TPS = 78 tok/s, i.e., 12.82 ms/output token) exhibit 130–470 ms end-to-end latency reductions despite input prefill ($\alpha_{\mathrm{prefill}} = 0.04$ ms/tok) saving only 2.0–3.3 ms directly: removing conversational fluff and anchoring invariants suppresses verbose output preambles (reducing mean generated output by 10.1–36.4 tokens and cutting verification retries $\bar{R}_k$ from 1.55 to 1.40, saving 36.4×12.82 ms ≈ 466.7 ms of Argon decode time). In Table III, *Compaction Only (on Gemini 4.0 Argon)* reports output-normalized cost (\$1.6586, isolating the exact \$0.10225 single-pass input token savings from 466.7 $\rightarrow$ 384.9 tokens; actual realized invoice cost including reduced output verbosity is \$1.2946), whereas in Table I, *Naive Regex Trimming* (\$1.5580) saves \$0.0525 on input tokens (-42.0 tok) plus \$0.1503 from premature output truncation (-15.03 output tok) caused by broken schema clauses (crashing Pass@1 to 89.20%).

All statistical prompt-compression baselines in Table III—LLMLingua [17], [18], LLMLingua-2 [19], and Selective Context [20]—were evaluated on Gemini 4.0 Argon at their standard $0.73\times$ –$0.80\times$ target retention ratios (342.0–373.3 tokens): token-level perplexity pruning drops schema delimiters and column keys, degrading Pass@1 to 74.20%–82.40% (even when LLMLingua-2 is constrained to a matched budget of exactly 384.9 tokens, Pass@1 reaches only 86.10%, and restricting LLMLingua strictly to unstructured instructions I in *Schema-Safe LLMLingua + Stage 1 AST* still drops negative modal qualifiers `never`/`not`, yielding 94.60% Pass@1). Ablating landmark sinks (*Compaction w/o Landmark Anchors*: 382.4 tokens, reflecting -7.0 landmark tokens and $+4.5$ retained section label tokens) drops Pass@1 from 100.00% to 95.80%. Standalone routers (FrugalGPT [21] and RouteLLM [22], calibrated on $N_{\mathrm{cal}} = 250$ to target $Q^{*} = 0.95$ across Flash and Argon) reach 97.40% and 98.60% Pass@1, whereas PromptVM's closed-form T_{think}^{*} allocation achieves the Pareto frontier: \$1.0663/1k, 99.70% first-attempt Pass@1, and 100.00% workflow completion.

Finally, to verify external generalization, we evaluated Stages 1–3 of PromptVM using stratified random sampling ($n = 200$ each, seed 42, $\tau_{\mathrm{temp}} = 0.0$) on two public benchmarks on fixed model tiers (Stage 5 routing disabled to isolate pure compaction fidelity): (1) on the **BIRD-SQL Dev** benchmark [13] (sampled across all database domains on fixed Gemini 4.0 Argon), PromptVM achieved 41.2% input token compaction (618.4 $\rightarrow$ 363.6 tokens), improved execution accuracy from 68.5% $\rightarrow$ 69.0%, reduced per-1k cost by 34.8%, and lowered mean latency by 18.4%; and (2) on the **Berkeley Function Calling Leaderboard (BFCL)** [2] (https://gorilla.cs.berkeley.edu/leaderboard.html, sampled across multi-tool AST categories on fixed Gemini 3.8 Flash), PromptVM achieved 38.4% token compaction (512.0 $\rightarrow$ 315.4 tokens), improved AST accuracy from 94.5% $\rightarrow$ 95.0%, and reduced cost by 36.1% with zero regressions.

VI. Related Work

Prompt Compression: Prior prompt compression works such as LLMLingua [17], LongLLMLingua [18], and Selective Context [20] filter tokens with low surprisal under small language models, while LLMLingua-2 [19] introduces task-agnostic data distillation. However, token-level statistical filtering frequently prunes negative polarity constraints, quotation delimiters, and numerical bounds, causing functional failures in structured tool and database tasks [2], [13]. In contrast, PromptVM provides deterministic, schema-grounded contract and parameter-role retention via AST transpilation and polarity-aware predicate signature tracking.

Attention Sinks, Modular KV-Cache Reuse, and Test-Time Compute Routing: StreamingLLM [14] demonstrated that initial sequence tokens act as natural attention sinks in autoregressive models. PromptVM establishes the complementary compile-time principle: synthetically injecting uppercase

TABLE II
DISAGGREGATED PERFORMANCE AND ANALYTICAL RECONCILIATION PARAMETERS BY ENTERPRISE COHORT ($n = 200$ EACH, 5 SEEDS)

Cohort	Raw → Comp. Tok.	Tok. Red.	$\hat{L}_{\text{out}}$ / Real $\hat{L}_{\text{out}}^{\text{real}}$	Turns $\hat{R}_k$ (Base→Ours)	Cache h	Flash / Argon Split	Base → Ours (/1k)	Cost Sav.	Latency: Model + Sandbox = Total	Pass@1 (1st / Wf)
Cohort A: SQL DB	273.1 → 156.1	42.84%	64 / 64.0 (208.2‡)	1.35 → 1.00	0.01	100.0% / 0.0% ($T^*_{\text{think}} = 0$)	\$1.3228 → \$0.1483	88.79%	831.2 + 714.2 = 1,545.4 ms	100.0% / 100.0%
Cohort B: OpenAPI Agent	274.2 → 167.3	38.98%	96 / 11.3	1.00 → 1.00	0.00	72.0% / 28.0% ($T^*_{\text{think}} = 8$)	\$0.4555 → \$0.3320	27.11%	798.4 + 655.0 = 1,453.4 ms	100.0% / 100.0%
Cohort C: Financial Compliance	236.0 → 174.0	26.27%	112 / 9.5	1.00 → 1.00	0.00	72.5% / 27.5% ($T^*_{\text{think}} = 16$)	\$0.3900 → \$0.2840	27.18%	812.9 + 641.0 = 1,453.9 ms	100.0% / 100.0%
Cohort D: Codebase Refactoring	1,348.0 → 1,333.0	1.11%	256 / 256.0 (94.0‡)	1.40 → 1.02	0.45	34.0% / 66.0% ($T^*_{\text{think}} = 128$)	\$5.9400 → \$4.0542	31.75%	1,064.6 + 482.0 = 1,546.6 ms	98.5% / 100.0%
Cohort E: Multi-Turn Reasoning	202.0 → 94.0	53.47%	80 / 44.3 (cum.)	3.00 → 3.00	0.42	74.0% / 26.0% ($T^*_{\text{think}} = 4$)	\$0.6957 → \$0.5130	26.26%	785.5 + 662.0 = 1,447.5 ms	100.0% / 100.0%
Pooled Mean ($N = 1{,}000$)	**466.7 → 384.9**	**17.52%**	**121.6 / 77.0**	**1.55 → 1.40**	**0.18**	**70.5% / 29.5%**	**\$1.7608 → \$1.0663**	**39.44%**	**858.5 + 630.9 = 1,489.4 ms**	**99.7% / 100.0%**

‡In Cohort A, Flash emits SQL plus structured audit metadata (208.2 tok); in Cohort D, compacted prompts emit unified diff patches (94.0 tok) vs. baseline full-file rewrites (256.0 tok).

structural landmark anchors bounds background attention dispersion while co-locating invariant clauses to amplify retrieval-head attention. In runtime memory management, vLLM [10] and SGLang [11] manage PagedAttention block tables and RadixAttention prefix reuse, while modular KV-cache reuse frameworks such as Prompt Cache [23] precompute reusable attention blocks with decoupled positional IDs; PromptVM provides the hardware-aligned complement that pads static prefixes to exact HBM2e page boundaries ($B \in \{16, 32\}$) with interior Pallas VMEM bitmasking to eliminate copy-on-write boundary-block duplication across concurrent sub-agents. Finally, in model routing and test-time compute scaling, systems such as FrugalGPT [21], Hybrid LLM [24], RouteLLM [22], and unified routing evaluation frameworks such as RouterBench [25] explore sequential cascading and preference-based tier selection, while recent difficulty-aware test-time compute scaling studies [26] show that allocating variable reasoning compute often outperforms scaling model parameters. PromptVM unifies these paradigms via a closed-form discrete MINLP that jointly selects model serving tiers and integer test-time reasoning budgets (T^*_{think}) co-designed with upstream prompt compaction.

VII. CONCLUSION

We presented the *Tokenomics Compiler* (`PromptVM`), a hardware-aware compilation and discrete-choice economic routing framework for enterprise LLM serving. By uniting tokenizer-aware AST transpilation, invariant density compaction, attention-sink landmark anchoring, server-side Pallas paged-cache boundary sealing, and discrete Pareto routing, our system slashes per-cohort enterprise inference costs by 26.26% to 88.79% (39.44% blended net savings), eliminates multi-branch copy-on-write boundary page duplication on Cloud TPU v6e, and achieves 99.70% strict first-attempt Pass@1 and 100.00% workflow completion while provably bounding attention dispersion.

Limitations and Trade-offs: Compaction efficiency is bounded by the ratio of structured schemas and boilerplate to dense payload content; on raw source code refactoring (Cohort D), AST schema compression yielded a 1.11% token reduction, with cost savings driven primarily by Stage 5 dynamic reasoning allocation. Future work will extend AST transpilation to programming language syntax trees via Tree-sitter parsers and integrate multi-tier host DRAM-to-HBM speculative page prefetching.

TABLE III
SYSTEM COMPONENT ABLATION MATRIX AND BASELINE COMPARISON ($N = 1{,}000$, 5 SEEDS)

System Configuration	Tokens	Cost/1k	Latency	Pass@1 (1st / Wf)
Monolithic Baseline (Gemini 4.0 Argon)	466.7	\$1.7608	2,450.0 ms	98.50% / 98.50%
Deterministic Schema Minification (RFC 8259 [8])	416.3	\$1.6980	2,320.0 ms	98.50% / 98.50%
LLMLingua Compression (Full Prompt, Argon) [17]	350.0	\$1.4700	2,120.0 ms	74.20% / 74.20%
LLMLingua-2 Distillation (Full Prompt, Argon) [19]	342.0	\$1.4480	2,080.0 ms	82.40% / 82.40%
Selective Context (Full Prompt, Argon) [20]	373.3	\$1.5280	2,190.0 ms	78.50% / 78.50%
Schema-Safe LLMLingua (I Only) + Stage 1 AST	398.2	\$1.5840	2,040.0 ms	94.60% / 94.60%
FrugalGPT Cascade [21]	466.7	\$1.2840	2,150.0 ms	95.80% / 97.40%
RouteLLM Preference Router (Raw) [22]	466.7	\$1.2180	1,680.0 ms	98.10% / 98.60%
Compaction + RouteLLM Router [22]	384.9	\$1.1120	1,540.0 ms	98.50% / 98.80%
Compaction Only (on Gemini 4.0 Argon)§	384.9	\$1.6586	1,980.0 ms	100.00% / 100.00%
Compaction w/o Landmark Anchors (on Argon)	382.4	\$1.6550	1,970.0 ms	95.80% / 95.80%
MINLP Routing Only (Raw Prompts)	466.7	\$1.3250	1,720.0 ms	97.50% / 98.00%
PromptVM Full Pipeline	**384.9**	**\$1.0663**	**1,489.4 ms**	**99.70% / 100.00%**

§Output-normalized cost isolating pure input savings (−\$0.10225); realized cost with shorter output is \$1.2946.

REFERENCES

[1] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Int. Conf. Learn. Represent. (ICLR)*, 2023.

[2] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large Language Model Connected with Massive APIs," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024.

[3] N. P. Jouppi *et al.*, "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings," in *Proc. ACM/IEEE 50th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2023, pp. 1–14.

[4] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 5, 2023, pp. 606–624.

[5] T. Kudo, "Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates," in *Proc. 56th Annu. Meet. Assoc. Comput. Linguist. (ACL)*, 2018, pp. 66–75.

[6] Gemini Team, Google, "Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context," *arXiv preprint arXiv:2403.05530*, 2024.

[7] Gemma Team, Google, "Gemma 2: Improving Open Language Models at a Practical Size," *arXiv preprint arXiv:2408.00118*, 2024.

[8] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," Internet Engineering Task Force (IETF), RFC 8259, Dec. 2017.

[9] A. Vaswani *et al.*, "Attention Is All You Need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[10] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. 29th ACM Symp. Oper. Syst. Princ. (SOSP)*, 2023, pp. 611–626.

[11] L. Zheng *et al.*, "SGLang: Efficient Execution of Structured Language Model Programs," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024.

[12] J. Bradbury *et al.*, "JAX: Composable Transformations of Python+NumPy Programs," 2018. [Online]. Available: http://github.com/google/jax

[13] J. Li *et al.*, "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.

[14] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," in *Int. Conf. Learn. Represent. (ICLR)*, 2024.

[15] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[16] Gemini Team, Google, "Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[17] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 13358–13376.
[18] H. Jiang *et al.*, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," in *Proc. 62nd Annu. Meet. Assoc. Comput. Linguist. (ACL)*, 2024, pp. 1658–1677.
[19] Z. Pan *et al.*, "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," in *Proc. 62nd Annu. Meet. Assoc. Comput. Linguist. (ACL)*, 2024, pp. 9635–9654.
[20] Y. Li, B. Dong, F. Guerin, and C. Lin, "Compressing Context to Enhance Inference Efficiency of Large Language Models," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 6342–6353.
[21] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance," *Trans. Mach. Learn. Res. (TMLR)*, 2024.
[22] I. Ong *et al.*, "RouteLLM: Learning to Route LLMs with Preference Data," in *Int. Conf. Learn. Represent. (ICLR)*, 2025.
[23] I. Gim *et al.*, "Prompt Cache: Modular Attention Reuse for Low-Latency Inference," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.
[24] D. Ding *et al.*, "Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing," in *Int. Conf. Learn. Represent. (ICLR)*, 2024.
[25] Q. J. Hu *et al.*, "RouterBench: A Benchmark for Multi-LLM Routing System," *arXiv preprint arXiv:2403.12031*, 2024.
[26] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters," *arXiv preprint arXiv:2408.03314*, 2024.